

Capstone Project Specification

Enterprise Quality Engineering with Playwright & Generative AI

Target Audience	Enterprise Client Automation Teams / SDETs	Language / Runtime	JavaScript (Node.js)
Framework Version	Playwright v1.4x+	Document Version	v1.0 (Comprehensive Proposal)

Executive Summary

This document establishes the structural, architectural, and evaluation blueprints for a comprehensive, production-grade Capstone Project. Designed for enterprise software quality engineering validation, this hands-on lab challenges participants to engineer a scalable, maintainable automation ecosystem. It seamlessly unifies advanced modern browser automation with state-of-the-art Generative AI testing paradigms, including Model Context Protocol (MCP) integrations.

1. Project Architecture Overview

The capstone project requires participants to architect a dual-project framework structure within a single unified multi-project layout. Rather than writing scattered test scripts, candidates will implement an enterprise-grade workspace that enforces structural modularity, clear separation of concerns, and robust configuration layers.

1.1 Structural Layout

- **Project #1: Enterprise Page Object Model (POM)** — Built targeting a highly dynamic, asynchronous enterprise micro-frontend application mimicking complex UI states, frames, data tables, and authentication boundaries.
- **Project #2: High-Performance SVG Graphics Engine Ecosystem** — Focused specifically on low-level DOM manipulations, shadow roots, and complex coordinate-based mathematical SVG object tracking.

2. Deep Dive: Implementation Milestones

Milestone 1: Environment Setup, CLI, & Core Framework Fundamentals

Participants must establish a clean Node.js workspace environment using explicit `npx` configurations. Global packages are explicitly prohibited to enforce sandbox execution consistency.

- Initialize a clean workspace and configure specific execution variants within `package.json` (e.g., `test:headed`, `test:parallel`, `test:debug`).
- Configure `playwright.config.js` to handle multiple execution profiles, dynamically utilizing `fullyParallel: true`, customized worker thresholds based on CPU availability, and retry logic for flakiness mitigation.
- Leverage UI Mode and the interactive `page.pause()` utility during script authorship to handle live debugging sessions within VS Code.

Milestone 2: DOM Navigation, Locators, & Dynamic Interactivity

This phase evaluates the engineer's capability to bypass brittle auto-generated paths and instead implement robust, resilient locator strategies.

- **Advanced Locator Strategies:** Standardize on user-facing semantic locators using `getByRole()` with explicit accessible names, cascading filters, and strict parent-child traversals to eliminate multi-element collisions.
- **Form Automation Mastery:** Design robust data-entry chains addressing standard text inputs, programmatic checkbox selections, explicit state validations for radio buttons, and complex dropdown components (handling both standard native HTML `<select>` tags and customized non-select custom listbox elements).
- **Data Handling & Hard/Soft Assertions:** Extract live layout text values, runtime attribute maps, and execution-state arrays. Integrate strategic `expect.soft()` barriers to verify peripheral visual metadata without halting critical test flow execution.

Milestone 3: Advanced Asynchronous & Complex Control Mechanisms

Modern applications present volatile lifecycles. This module addresses advanced UI barriers that cause conventional automation tools to fail.

- **Table Automation:** Extract, sort, and validate live grid systems. Candidates must parse structured tabular data dynamically, handle multi-page pagination controls, and dynamically click active row-actions mapped against targeted data records.
- **Shadow DOM & Iframe Isolation:** Construct selectors capable of piercing open/closed Shadow roots. Handle multi-tiered cross-domain security boundaries using `frameLocator()` structures.

- **Low-Level Interactions & File Systems:** Synthesize complex, native hardware events through keyboard layouts and pointer sequences (such as drag-and-drop actions, hover cards, and multi-file upload streams via asynchronous system buffers).
- **Dialogs & Window Routing:** Intercept and handle native browser dialog popups (alerts, prompts, confirms) using automated event listeners, alongside multi-tab parent-child context window lifecycle tracking.

Milestone 4: Resilient Infrastructure, Synchronization, & Enterprise Reporting

Focuses on enterprise operational stabilization, establishing absolute transparency on framework failures across continuous delivery pipelines.

- **Auto-waiting Mechanics:** Configure granular actionability states and explicitly override timeout parameters strictly where required, avoiding arbitrary thread sleeps.
- **Visual Regression & Masking:** Implement pixel-perfect visual testing. Apply structural element masking to redact highly dynamic runtime fields (e.g., timestamps, randomized unique IDs) to ensure visual snapshots remain deterministic across execution cycles.
- **Artifact Collection & Triaging:** Setup dynamic context triggers to capture video streams, high-resolution screenshots on failure, and complete multi-threaded zip traces for analysis via the Playwright Trace Viewer.
- **Reporting:** Build a multi-tier dashboard blending standard HTML reports with deep Allure Report historical analytics summaries.

Milestone 5: Programmable API Operations & Mocking Strategies

Eliminate front-end automation dependencies by injecting targeted API-level interactions.

- Implement direct programmatic API calls using Playwright's built-in `request` context to bypass standard authentication workflows, dramatically speeding up test pre-conditions.
- Construct request interceptors via `page.route()` to mock backend responses. Deliver custom JSON payloads, simulate negative edge-case status codes, and test front-end UI resilience against backend service dropouts or high-latency responses.

Milestone 6: Modern Generative AI Engineering & MCP Integration

This cutting-edge module upgrades the traditional QA engineer into an AI-augmented Quality Architect, validating practical skills with Next-Gen AI toolsets.

- **IDE Acceleration:** Provision GitHub Copilot inside VS Code to accelerate test generation, construct boilerplate data factories, and refactor brittle CSS structures into clean semantic locators.
- **Model Context Protocol (MCP):** Configure an active MCP host environment inside the workspace, exposing the local system architecture, DOM state, and Playwright execution runtime directly to an LLM engine.

- **Autonomous Test Scenario Engineering:** Use MCP-driven workflows to analyze application state, interpret live system shifts, and auto-generate end-to-end regression validation suites dynamically based on real-time feedback loops.

3. Capstone Deliverables & Evaluation Matrix

To successfully complete the qualification, engineers must submit a production-ready repository matching the criteria below. Reviewers will evaluate projects using a strict enterprise architectural checklist:

Evaluation Domain	Success Criteria	Weightage
Framework Design & POM	Clean folder structuring; distinct page layer separating selectors/actions from execution specs; strict custom fixtures usage over generic hooks.	25%
Resiliency & Synchronization	Zero flakiness; complete absence of hard-coded wait commands; exact use of auto-waiting and tailored custom state assertions.	20%
Complex DOM Handling	Accurate extraction of multi-tier iframe elements, deep Shadow DOM piercing, and precise mathematical SVG component interactions.	20%
API & Mocking Excellence	UI speedups via injected API authentication setups; fully simulated edge-case conditions via custom route masking.	15%
AI & MCP Innovation	Demonstrated prompt files, documented MCP configuration mapping, and robust AI-assisted regression spec generation scripts.	10%
CI/CD & Reporting Insight	Flawless local/headless Jenkins execution logs; multi-platform execution configuration; complete Allure visual history trace output.	10%

4. Final Review Checklist for Clients

Before delivering or presenting the final repository workspace to the client, ensure the code satisfies the following engineering rules:

- [] All parameters are entirely data-driven via external JSON or environment configurations. No credentials or base URLs are hardcoded inside page classes.

- [] Every single test block executes independently. Parallel execution runs flawlessly across multiple workers without cross-test session pollution.
- [] Custom locator strategies rely primarily on accessible semantic names (`getByRole`) or stable data attributes (`data-testid`) rather than auto-generated structural XPaths.